Agent-Based Simulation of a Sortation Warehouse in NetLogo Web

A more natural and reader-friendly version of the project report

Harish Ulaganathan Indhumathi

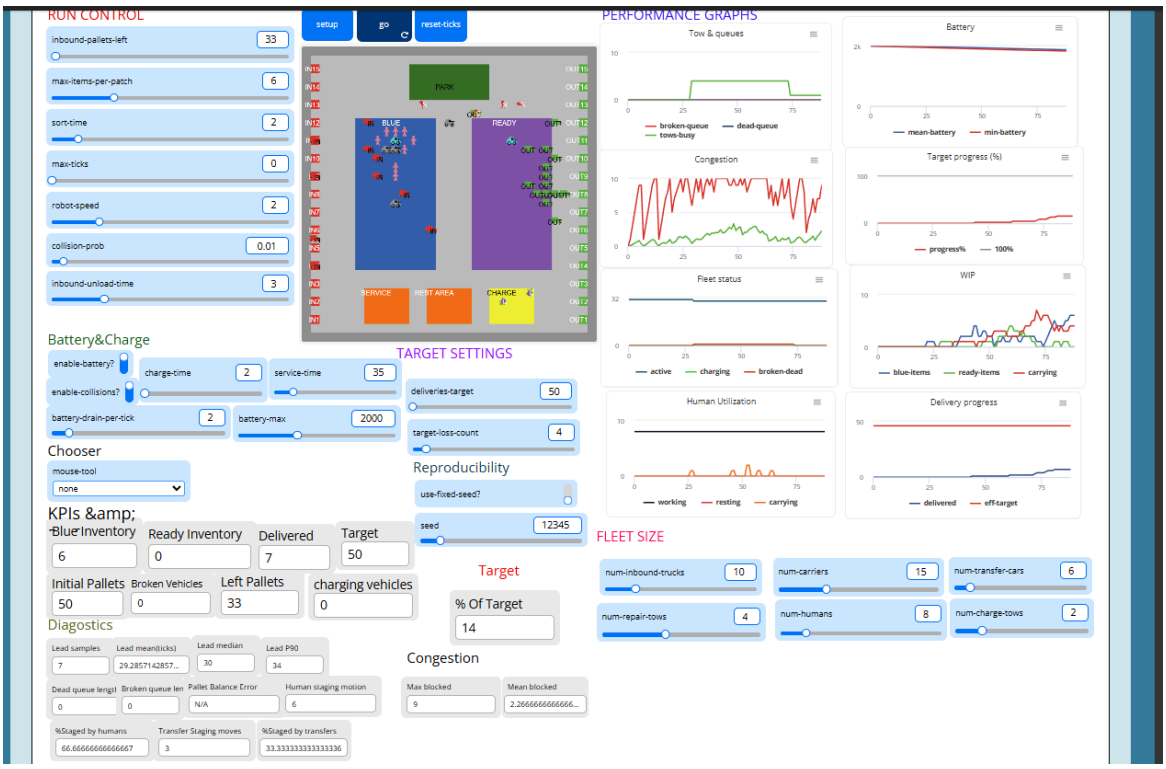

Figure 1. Main NetLogo Web interface showing warehouse zones, controls, KPIs, and performance charts.

Course: Agent Based Modelling and Parallel Computing
This version keeps the technical meaning of the original work while making the language smoother, clearer, and more human to read.

Abstract

This project explores how a sortation warehouse behaves when people, automated vehicles, and support fleets all have to share the same workspace. In the model, pallets arrive at the inbound side, are staged in the BLUE area, moved into the READY buffer, and then dispatched through outbound doors. What makes the problem interesting is that this flow is not smooth by default. It is shaped by congestion in narrow corridors, blocking between agents, battery limits, repair downtime, and the simple fact that different parts of the system can become bottlenecks at different times.

The warehouse was implemented in NetLogo Web as an agent-based model with six main agent types: inbound trucks, transfer cars, outbound carriers, humans, repair tows, and charge tows. A staged movement rule - plan, resolve, and commit - was used to keep shared-space movement stable inside NetLogo Web's single-threaded runtime.

The results show a clear pattern: performance depends less on the total number of agents than on how well the stages of the warehouse stay balanced. When inbound flow is too high, the BLUE zone fills and congestion rises. When outbound capacity becomes the limit, items collect in READY. This makes the model useful as a practical and visual way to test operational ideas before trying them in a real warehouse.

Table of Contents

Abstract 2

Table of Contents 3

1. Introduction 4

1.1 Problem statement . . . 4

1.2 Objectives . . . 4

1.3 Scope . . . 4

2. Model Design 4

2.1 Warehouse layout and zones . . . 4

2.2 Agent types and roles . . . 5

2.3 Behavior, rules, and interactions . . . 5

2.4 Effective delivery target . . . 5

3. Implementation in NetLogo Web 6

3.1 Runtime model and logical concurrency . . . 6

3.2 Plan - resolve - commit movement barrier . . . 6

3.3 System architecture . . . 6

3.4 Interface controls and monitoring . . . 7

4. Experiments and Results 7

4.1 Scenario design . . . 7

4.2 Sample run summary . . . 8

4.3 Visual evidence from model outputs . . . 9

5. Discussion and Business Insights 9

5.1 Interpreting the warehouse behavior . . . 9

5.2 Reliability and recovery . . . 10

5.3 Why this matters in practice . . . 10

6. Limitations and Future Work 10

7. Conclusion 10

1. Introduction

Warehouses look simple from a distance: goods come in, goods move around, and goods go out. In practice, they are messy shared systems. People walk through the same corridors that vehicles use, batteries run low at inconvenient times, and one blocked lane can slow several processes at once. This project was built to understand that reality in a more structured way.

The model focuses on a sortation warehouse where pallets are unloaded inbound, staged and processed, and then moved onward for delivery. Instead of treating the warehouse as one average system, the simulation breaks it into agents with their own roles and behaviors. That choice matters because many operational problems are caused by interaction rather than by one isolated machine or one isolated worker.

1.1 Problem statement

The core problem is to understand why warehouse performance falls short even when there seems to be enough work and enough equipment. The original report points to familiar causes: congested corridors, contested movement, downtime from charging or service, and failures that spread delays beyond the affected vehicle. The model asks how these frictions change throughput and the ability to reach delivery targets.

1.2 Objectives

- Study how inbound supply, transfer capacity, and outbound delivery interact.
- Use work-in-progress patterns to show where bottlenecks are forming.
- Observe how humans and autonomous vehicles coordinate in a shared layout.
- Measure the effect of congestion, collisions, battery drain, and recovery delays.
- Provide a practical experiment space for policy testing before real operational changes are made.

1.3 Scope

The model focuses on internal warehouse flow and disruption handling. It includes warehouse zones, patch capacities, finite inbound supply, transport logic, charging and service recovery, and system-level monitoring. It does not attempt to model everything. Conveyor details, SKU-level order logic, full vehicle physics, detailed human ergonomics, and external road traffic are left out so the simulation can remain clear and responsive.

2. Model Design

2.1 Warehouse layout and zones

The warehouse is represented as a patch grid with named zones. The main operational spaces are INFEED, BLUE, READY, OUTFEED, SERVICE, CHARGE, PARK, and REST. This layout was not chosen only for appearance. It creates shared corridors, handoff points, and pressure areas where queues and bottlenecks can emerge naturally from agent interaction.

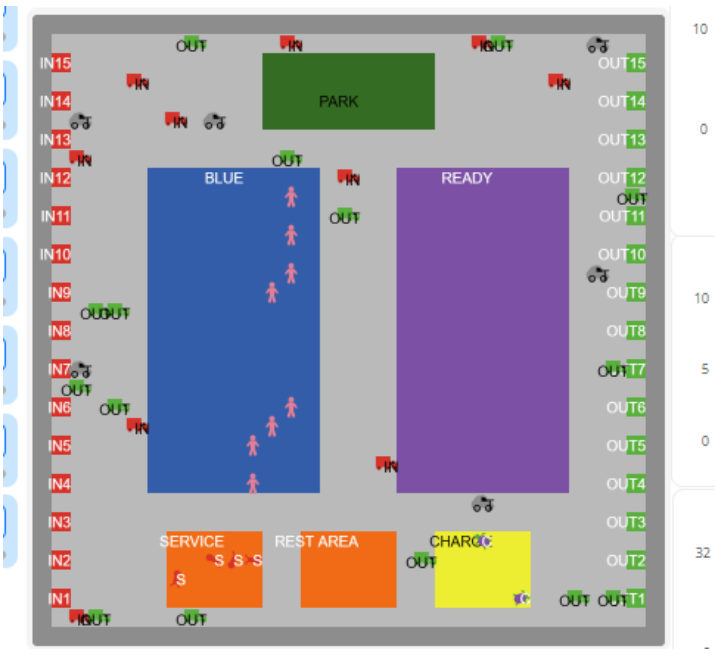

Figure 2. Simplified warehouse layout showing the main operational zones used in the model.

2.2 Agent types and roles

Each agent has a clear role in the warehouse flow. Inbound trucks introduce pallet supply into BLUE. Transfer cars move items from BLUE to READY. Outbound carriers move items from READY to the outfeed side. Humans represent sorting and support work. Repair tows and charge tows handle recovery when vehicles fail or lose battery. This mix makes the model feel closer to a real warehouse, where productive work and support work are tightly linked.

2.3 Behavior, rules, and interactions

The model uses state-based behaviors instead of instant movement. Unloading, sorting, charging, and service all take time. Vehicles move patch by patch, so congestion is not imposed from outside; it emerges when several agents need the same space. Humans also follow a give-way rule near transfer activity, which reduces blocking in crowded areas. Collisions are simplified, but they still matter because a broken or depleted vehicle can create secondary delays across the system.

2.4 Effective delivery target

One of the most sensible design choices in the project is the effective target. The model clamps the delivery target to what can realistically be shipped from the inbound supply that actually entered the system. That avoids a misleading situation where a scenario appears to fail only because not enough pallets were available in the first place.

3. Implementation in NetLogo Web

3.1 Runtime model and logical concurrency

NetLogo Web runs in a single execution thread, so the project does not claim true operating-system multi-threading. Instead, it uses logical concurrency. In simple terms, warehouse activities such as unloading, sorting, vehicle movement, delivery, and recovery are scheduled every tick in a coordinated order. This creates the feel of many things happening at once while still keeping shared resources consistent.

3.2 Plan - resolve - commit movement barrier

The movement barrier is one of the strongest ideas in the model. Agents first plan their next move, then conflicts over contested patches are resolved, and only then are moves committed. This greatly reduces unfair ordering effects and makes shared-space movement more believable, especially in busy corridors.

3.3 System architecture

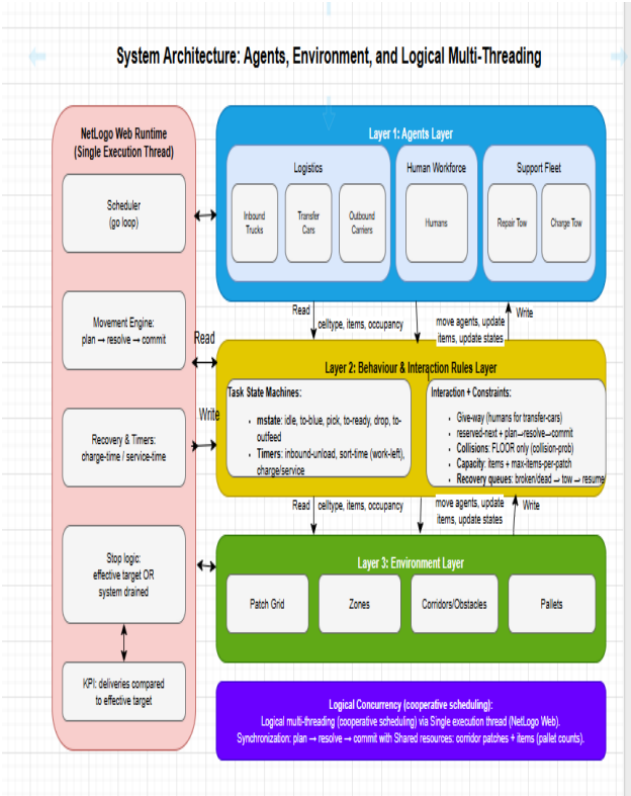

Figure 3. System architecture linking the runtime layer, agents layer, behavior layer, environment layer, and logical concurrency approach.

The architecture organizes the model into three layers. The agents layer contains logistics agents, humans, and support fleets. The behavior layer holds state machines, timers, rules, and constraints. The environment layer holds the grid, zones, corridor logic, and pallet counts. This is a clean design because it separates who acts, how they act, and where they act.

3.4 Interface controls and monitoring

The interface is designed for experimentation. Buttons such as setup, go, and reset-ticks handle execution. Sliders adjust inbound supply, robot speed, collision probability, sorting time, battery drain, charging time, service time, target settings, and fleet size. The graphs then show whether performance problems are coming from congestion, work-in-progress, battery pressure, tow activity, or weak delivery progress.

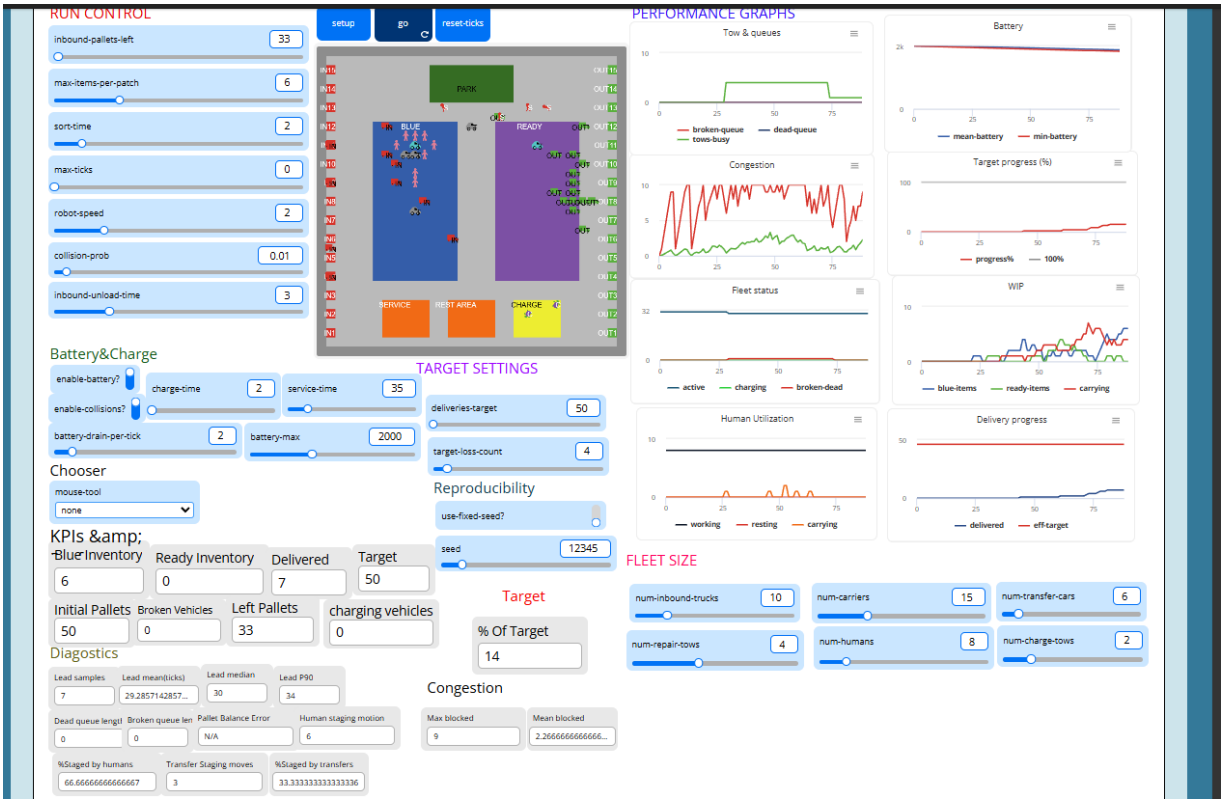

Figure 4. Main interface used to configure scenarios and observe warehouse performance during a run.

4. Experiments and Results

4.1 Scenario design

The original project explored several useful scenario families: a balanced baseline, inbound overload, outbound bottleneck, congested layout, and battery stress with towing. These scenarios are valuable because they help separate different kinds of failure. Sometimes the problem is too much inbound work. Sometimes the problem is a weak outbound side. Sometimes the real issue is simply too much traffic in too little space.

- Inbound overload pushes more pallets into BLUE than transfer and sorting can absorb.

- Outbound bottlenecks cause READY to fill because dispatch capacity becomes the limit.

- Congested layouts increase blocked time and force more waiting and replanning.

- Battery stress reduces the active fleet and increases demand for tow and charging support.

4.2 Sample run summary

Metric	Value
Final deliveries	50
Effective target	50
Ticks to target	2006
Average congestion	2.592
Peak congestion	10
Peak dead vehicles	0
Peak broken vehicles	0

In this sample run, the model reached the effective target with moderate average congestion. That does not mean the warehouse was frictionless. Local peaks still appeared, which suggests that a system can look stable overall while still experiencing short bursts of operational stress.

4.3 Visual evidence from model outputs

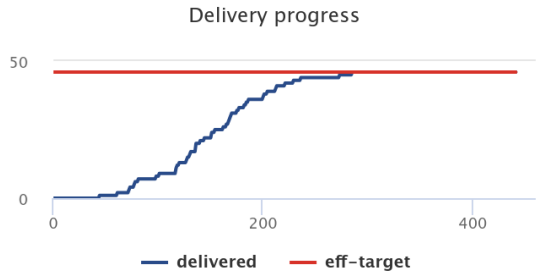

Figure 5. Delivery progress against the effective target.

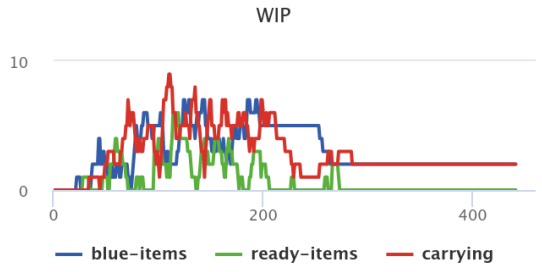

Figure 6. Work-in-progress across BLUE, READY, and carried items.

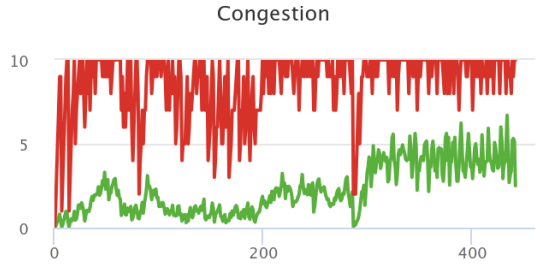

Figure 7. Congestion trends showing mean and peak blocking.

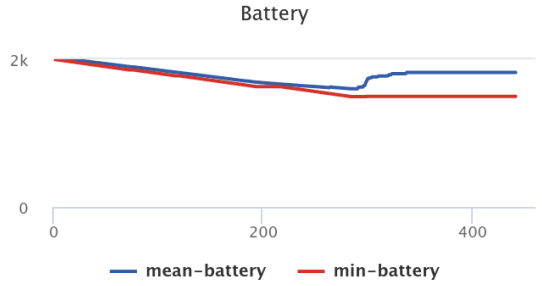

Figure 8. Mean and minimum battery levels over time.

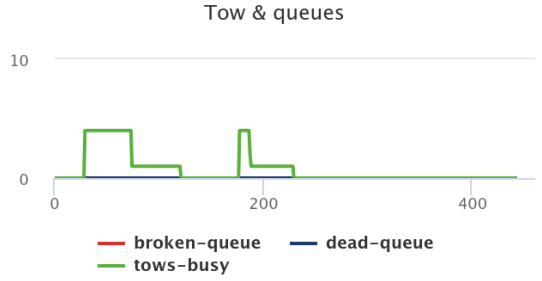

Figure 9. Tow utilization and queue behavior.

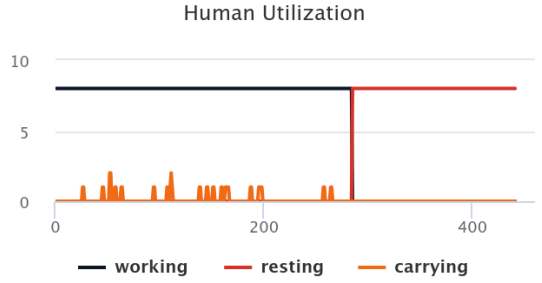

Figure 10. Human utilization across working, resting, and carrying states.

Together, these plots tell a story. Delivery rises steadily, WIP shifts between zones, congestion spikes when movement becomes contested, and recovery activity shows how supporting fleets affect smooth operation. The visuals make the model easier to explain because they connect warehouse behavior to observable system patterns.

5. Discussion and Business Insights

5.1 Interpreting the warehouse behavior

The biggest lesson from the model is simple: warehouse output depends on the slowest stage, not just on how many agents are in the system. If BLUE fills up while READY stays low, the pressure is around unloading, transfer, or sorting. If READY starts filling instead, then the outbound side is likely the bottleneck. That makes work-in-progress location a very practical signal for diagnosis.

Congestion is another important insight. A warehouse can have enough pallets and enough vehicles on paper and still perform poorly if shared corridors become crowded. In other words, capacity is not only about asset count; it is also about how easily those assets can move.

5.2 Reliability and recovery

The project also shows that failures have ripple effects. A broken or battery-depleted vehicle does not only remove one unit of capacity. It can block travel lanes, consume tow resources, increase waiting elsewhere, and delay other jobs until recovery is complete. This is why charging areas, service capacity, and support fleets deserve more attention than they often receive in simple warehouse models.

5.3 Why this matters in practice

For someone managing operations, the takeaway is clear: adding more robots is not always the answer. Sometimes the better intervention is a cleaner corridor design, faster recovery, better charging policy, or stronger balance between inbound and outbound stages. The model is useful because it gives a safe way to test these ideas before they become real operational decisions.

6. Limitations and Future Work

This model intentionally keeps some things simple so that the system stays readable and runs well in the browser. Pallets are stored as counts on patches rather than as individual pallet agents. Movement is grid-based rather than physics-based. Conveyor detail, SKU-level order logic, detailed human ergonomics, cost modeling, and external road effects are outside the current scope.

- Add richer routing and dispatch rules in dense corridor conditions.
- Introduce pallet priorities or order classes to study service differentiation.
- Expand charging and service policies to compare recovery strategies.
- Add cost and energy metrics for stronger trade-off analysis.
- Move toward a digital-twin style workflow with broader scenario validation.

7. Conclusion

At its heart, this project is about making warehouse behavior easier to see and reason about. By linking inbound supply, staging, sorting, outbound dispatch, congestion, and recovery in one model, it shows how local interactions grow into system-level outcomes such as queue buildup, throughput loss, and recovery-driven slowdowns.

What makes the model valuable is not only that it runs, but that it explains. It helps distinguish whether shortfalls come from limited supply, transfer pressure, weak outbound capacity, or movement friction inside the warehouse. That makes it a strong foundation for future academic work and a credible starting point for more practical decision-support experiments.